```
  has_many :orders do
    def find_by_order_prefix(order_number)
      find_by(region_id: order_number[0..2])
    end
  end
end
```

If you have an extension that should be shared by many associations, you can use a named extension module. For example:

```
module FindRecentExtension
  def find_recent
    where("created_at > ?", 5.days.ago)
  end
end

class Customer < ActiveRecord::Base
  has_many :orders, -> { extending FindRecentExtension }
end

class Supplier < ActiveRecord::Base
  has_many :deliveries, -> { extending FindRecentExtension }
end
```

Extensions can refer to the internals of the association proxy using these three attributes of the `proxy_association` accessor:

- `proxy_association.owner` returns the object that the association is a part of.
- `proxy_association.reflection` returns the reflection object that describes the association.
- `proxy_association.target` returns the associated object for `belongs_to` or `has_one`, or the collection of associated objects for `has_many` or `has_and_belongs_to_many`.

# 5 Single Table Inheritance

Sometimes, you may want to share fields and behavior between different models. Let's say we have Car, Motorcycle and Bicycle models. We will want to share the `color` and `price` fields and some methods for all of them, but having some specific behavior for each, and separated controllers too.
Rails makes this quite easy. First, let's generate the base Vehicle model:

```
$ rails generate model vehicle type:string color:string
price:decimal{10.2}
```

Did you note we are adding a "type" field? Since all models will be saved in a single database table, Rails will save in this column the name of the model that is being saved. In our example, this can be "Car", "Motorcycle" or "Bicycle." STI won't work without a "type" field in the table.

Next, we will generate the three models that inherit from Vehicle. For this, we can use the `--parent=PARENT` option, which will generate a model that inherits from the specified parent and without equivalent migration (since the table already exists).
For example, to generate the Car model:

```
$ rails generate model car --parent=Vehicle
```

The generated model will look like this:

```
class Car < Vehicle
end
```

This means that all behavior added to Vehicle is available for Car too, as associations, public methods, etc.